# Report on Mini Project

**Subject: Big Data Analytics (CSC702)**

**AY: 2026–27**

# MovieLens Big Data Analytics and Recommendation Dashboard

**Sarthak Agrawal: 2303002**

**Ahmed Ansari: 2303009**
**Sunil Bhagat: 2303020**
**Yash Bhardwaj: 2303021**

**Guided by**

**Prof. Anagha Durugkar**

# CHAPTER 1: INTRODUCTION

## Project Description

This project presents a MovieLens Big Data Analytics dashboard that brings together movie metadata, statistical visualizations, query results, and a basic genre-based recommendation feature in one web interface. The dashboard is implemented using Python and Flask, with MySQL as the main application database. It is intended to make movie data easier to explore and to demonstrate how data ingestion, storage, querying, analytics, and presentation can fit into a big-data workflow.

The dashboard displays high-level dataset indicators, including the total number of movies, average available rating, number of languages represented, and number of release years. It also provides visualizations of movie releases over time, genre combinations, and the distribution of ratings. A top-rated movies table highlights titles with at least 1,000 votes. A separate Hive Analytics section allows users to inspect stored outputs from queries that were executed in Apache Hive on HDFS.

The project additionally includes a lightweight content-based recommendation feature. A user selects a movie, and the application finds other titles that share one or more genres. Candidates must have a non-null rating and at least 100 votes; the application returns up to five results ordered by rating and then vote count. This feature demonstrates a practical use of the dataset without claiming to be a trained machine-learning model.

## Questions Answered Through the Project

- What is the overall size and basic composition of the movie dataset?
- How does the number of movies vary by release year?
- Which genre combinations occur most frequently in the stored data?
- How are movie ratings distributed across the dataset?
- Which movies have high ratings while meeting a minimum vote threshold?
- How can movies be recommended using genre overlap and existing ratings?
- How can query results produced in Hive be presented in a dashboard for review?

## Market Betterment

The dashboard can support exploratory work for movie catalog teams, content analysts, researchers, and students. Genre and release-year summaries can help users understand the composition of a catalog. Rating and vote counts provide context when comparing titles, while the recommendation feature can help users discover movies with related genres. The project is primarily an academic prototype; operational use would require data-source documentation, stronger validation, and regular data refreshes.

## Tools and Technologies Used

- Python 3 and Flask for application logic and web rendering.
- MySQL and mysql-connector-python for storing and querying movie metadata.
- Matplotlib for dynamically generated dashboard charts.
- R for generating the rating-distribution visualization and other statistical output images.
- Apache Hadoop HDFS, Apache Hive, and Sqoop as part of the project's big-data workflow.
- HTML and CSS, rendered through Flask/Jinja templates, for the dashboard interface.

## User Interface and Visualization

The user interface is a single-page dashboard with KPI cards, charts, a top-rated movie table, a Hive Analytics query selector, and a movie recommendation selector. The design uses separate cards and tables to keep results readable and to make the workflow suitable for a short live demonstration.

# CHAPTER 2: DATA DESCRIPTION AND ANALYSIS

## Dataset Overview

The application's MySQL database is named movielens and contains a movies table. The dashboard reports 10,000 movie records. The fields used by the application include movie_id, title, original_title, release_year, runtime_minutes, genres, language, rating, num_votes, and is_adult. The application code and screenshots establish these fields and the displayed row count; the exact original dataset publisher and its licensing details should be confirmed from the downloaded dataset documentation before final submission.

## Data Format and Structure

- Movie identification: movie_id, title, original_title.
- Release information: release_year and runtime_minutes.
- Classification: genres and language.
- Rating information: rating and num_votes.
- Additional metadata: is_adult.

## Data Analysis Performed

The dashboard calculates four summary indicators directly from MySQL: total rows in the movies table, average non-null rating, distinct non-null languages, and distinct non-null release years. The captured dashboard shows 10,000 movies, an average rating of 6.12, 37 distinct language values, and 121 distinct release-year values.

The release-year chart groups records by release year and visualizes the number of movies for each year. The genre chart groups the genres field by its stored genre combination and displays the most common combinations. Because the genres field may contain combinations such as Comedy,Drama, the chart represents stored combinations rather than a fully normalized count of every individual genre.

The rating-distribution figure is generated using R and displayed in the Flask dashboard as an image. The top-rated table filters for non-null ratings and at least 1,000 votes, then sorts by rating and vote count. The recommendation feature uses a lower minimum threshold of 100 votes.

## Important Interpretation Notes

- The average rating is calculated over rows with a non-null rating; it is not a personalized score.
- Vote counts provide context, but the current ranking does not calculate a statistical confidence-adjusted rating.
- The dashboard shows 37 distinct language values, including coded or unknown values as represented in the source data.
- The displayed release-year range can include incomplete recent years; the latest year should not be interpreted as a full-year count.
- The Hive Analytics section displays saved results from previously executed Hive queries. It should be described as stored Hive output, not a live Hive connection from the web request.

# CHAPTER 3: DESIGN OF DATA PIPELINE

## Data Pipeline Architecture

The project workflow is organized as a sequence of data preparation, storage, analysis, and presentation stages. The exact command-level configuration can vary by the local environment, but the project combines a MySQL-backed web dashboard with outputs from the Hadoop/Hive workflow.

1. Data source and preparation: movie metadata is made available in a structured tabular form and loaded into the MySQL movies table.
2. Big-data processing: Hadoop HDFS and Apache Hive are used in the project workflow to store/query data; Sqoop is part of the data-transfer toolset.
3. Analytics: SQL queries calculate aggregate statistics and identify highly rated titles; R produces statistical visualization output.
4. Application layer: Flask queries MySQL, reads the saved Hive result JSON file, and passes results to the HTML template.
5. Presentation layer: the browser renders KPI cards, chart images, tables, Hive query results, and recommendations.

## Pipeline Component Explanation

### MySQL Database

MySQL acts as the operational data source for the dashboard. The Flask application uses parameterized queries for the selected movie and retrieves the fields needed for statistics, charts, the top-rated list, and recommendations.

### Flask Web Application

Flask routes the browser request to the dashboard function. The function retrieves aggregate metrics, the top-rated movies, movie choices for the recommendation selector, and the chosen Hive result. Jinja templates render the returned values into the page.

### Charts and R Output

Matplotlib is used to produce charts dynamically for release-year and genre summaries. The rating-distribution image is generated separately by R and served as a static image. The page groups these visualizations together for comparison.

### Hive Results

The Hive Analytics panel reads hive_results/hive_results.json and lets the user select among saved query outputs. It shows each query's title, SQL text, row count, and result table. This makes the results demonstrable without requiring the dashboard to start a Hive query every time a user changes the selection.

### Recommendation Logic

The recommendation logic retrieves the selected movie's genres, splits the comma-separated values, and searches for other movies containing at least one of those genres. It excludes the selected movie, requires a rating and at least 100 votes, sorts by rating descending and then vote count descending, and returns at most five results. This is a simple content-based baseline; it does not use user histories, collaborative filtering, or model training.

## Data Flow Summary

Structured movie data → MySQL and Hadoop/Hive workflow → SQL/R analysis → Flask dashboard → interactive tables and recommendations.

## Design Considerations

- Keep database access in reusable query functions and use parameterized queries for user-selected values.
- Use minimum vote thresholds to reduce the visibility of titles with very few ratings.
- Keep saved Hive results clearly labelled so users can distinguish them from live database queries.
- Treat the recommendation output as genre similarity rather than a personalized prediction.

# CHAPTER 4: RESULT ANALYSIS

## Dashboard Overview

The screenshots below document the implemented dashboard and its main outputs. The dashboard screenshot reports 10,000 movies, an average rating of 6.12, 37 languages, and 121 release years. The charts summarize release-year counts, genre combinations, and rating distribution.

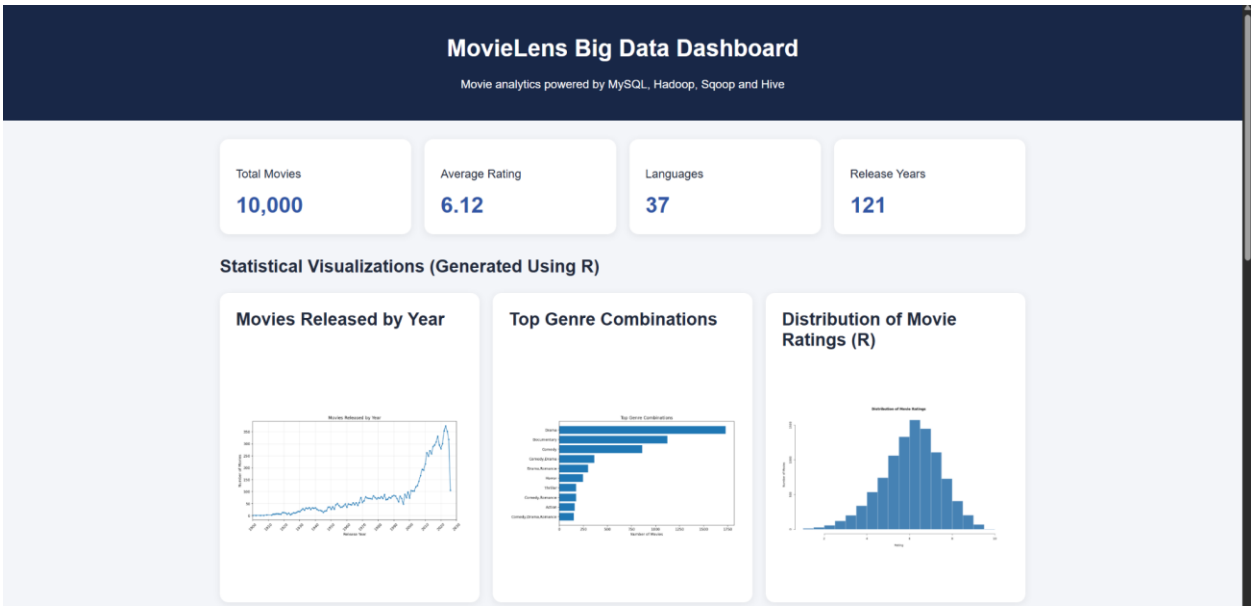



*Figure 4.1: MovieLens dashboard summary cards and statistical visualizations.*

## Top Rated Movies

The top-rated table applies a minimum threshold of 1,000 votes. In the captured output, Time of Violence appears with a rating of 9.0 and 2,627 votes. Other displayed titles include Gintama: The Final (8.7), Louis Tomlinson: All of Those Voices (8.6), and several titles rated 8.5. These are the results shown by the dataset at the time of capture and should not be presented as a universal or current ranking.



Top Rated Movies (minimum 1,000 votes)

| Title | Year | Rating | Votes |
|---|---|---|---|
| Time of Violence | 1988 | 9.0 | 2627 |
| Gintama: The Final | 2021 | 8.7 | 4244 |
| Louis Tomlinson: All of Those Voices | 2023 | 8.6 | 2123 |
| The Lion King | 1994 | 8.5 | 1275238 |
| The Intouchables | 2011 | 8.5 | 1040226 |
| The Kashmir Files | 2022 | 8.5 | 579760 |
| Methagu | 2021 | 8.5 | 8939 |
| Kaalam raasina kathalu | 2024 | 8.5 | 3996 |
| While We Watched | 2022 | 8.5 | 2894 |
| Brigade Miscellaneous Steps In | 1970 | 8.5 | 2094 |

*Figure 4.2: Top-rated movies table, filtered to movies with at least 1,000 votes.*

## Hive Analytics Results

The Hive Analytics panel presents saved outputs from queries executed in Apache Hive on HDFS. The top-rated query selects title, release year, rating, and vote count; it excludes null ratings, requires at least 1,000 votes, sorts by rating and vote count, and limits the output to ten rows.

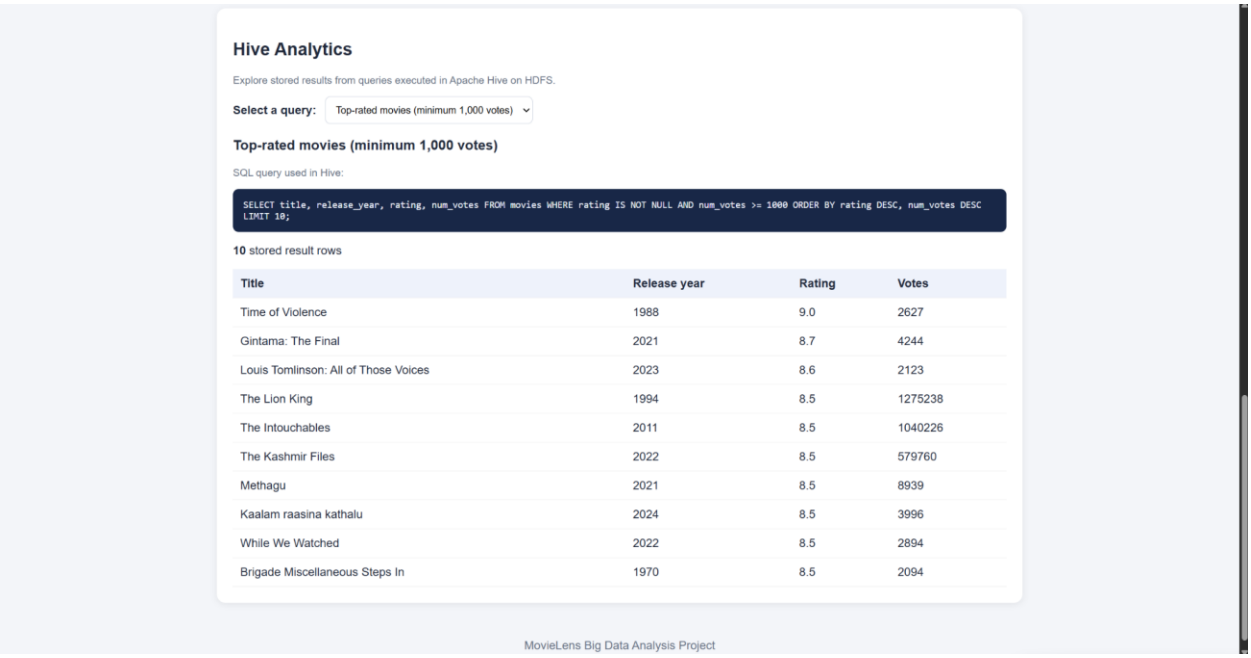



*Figure 4.3: Stored Hive result for the top-rated movies query.*

A second stored query groups records by language, calculates movie counts and average ratings, sorts by movie count, and shows up to 15 rows. In the screenshot, the `en` language code has 6,583 movies in the query output, followed by `Unknown` with 2,452. These counts describe the stored query result and its filtering conditions.

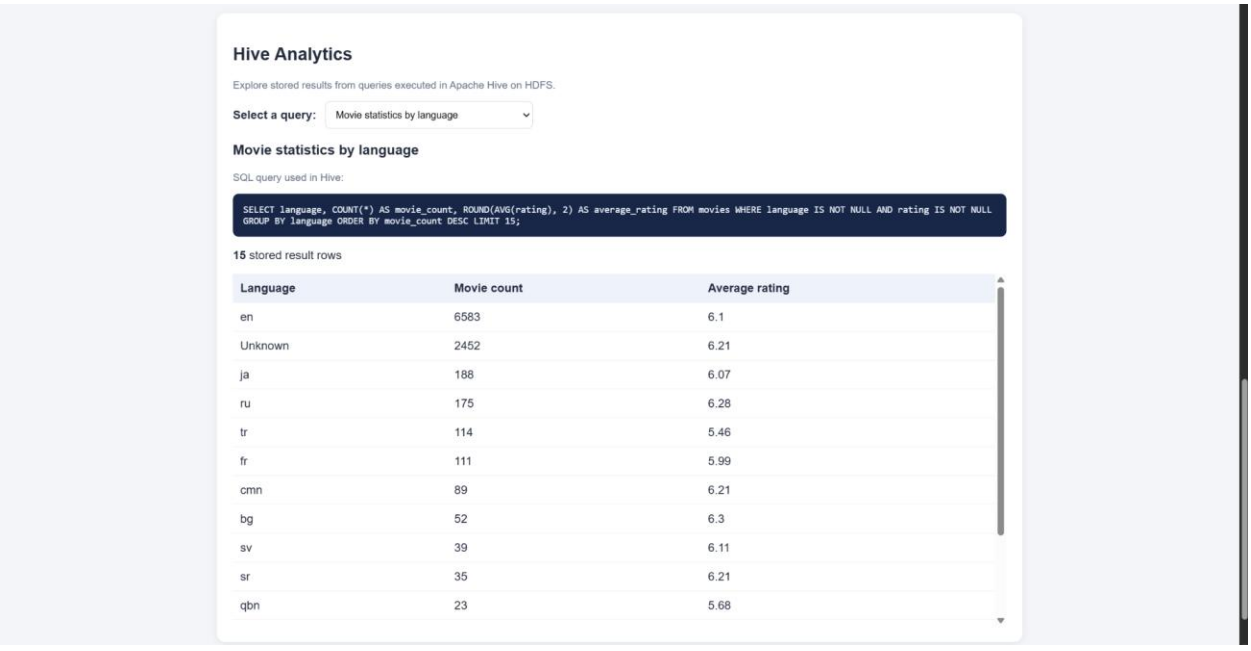



*Figure 4.4: Stored Hive query output for movie statistics by language.*

## Terminal Execution Evidence

The following terminal screenshots document the project directory, the HDFS setup, and a successful Hive query execution with its returned results.

```
platinum@Platinum-Pavilion:~$ cd ~/movielens_bigdata_project
ls
Hadoop-Project-master  derby.log          r_outputs
__pycache__            hive_results       results.txt
add_recommender.py     import_movies.py   sqoop-1.4.7.bin__hadoop-2.6.0
app.py                 movie_analysis.R   sqoop-1.4.7.bin__hadoop-2.6.0.tar.gz
app.py.backup          movies.java
app.py.before_hive     movies_for_r.tsv
platinum@Platinum-Pavilion:~/movielens_bigdata_project$
```

*Figure 4.6: MovieLens Big Data project directory and supporting files.*

```
platinum@Platinum-Pavilion:~/movielens_bigdata_project$ ~/hadoop-3.3.6/bin/hdfs d
fs -ls /
2026-10-15 12:20:57,105 WARN util.NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where applicable
Found 4 items
drwxr-xr-x   - platinum supergroup          0 2026-10-11 16:04 /movie_ott_project
drwxr-xr-x   - platinum supergroup          0 2026-10-14 08:42 /movielens_project
drwx-wx-wx   - platinum supergroup          0 2026-10-11 16:11 /tmp
drwxr-xr-x   - platinum supergroup          0 2026-10-11 15:58 /user
platinum@Platinum-Pavilion:~/movielens_bigdata_project$
```

*Figure 4.7: HDFS root directory showing the project and user directories.*

```
hive> SELECT title, release_year, rating
    > FROM movies
    > WHERE rating IS NOT NULL
    > ORDER BY rating DESC
    > LIMIT 5;
Query ID = platinum_20261015130322_10a0abdb-9553-4571-9c65-302d00bc6543
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1792049490106_0002, Tracking URL = http://Platinum-Pavilion.localdomain:8088/proxy/application_1792049490106_0002/
Kill Command = /home/platinum/hadoop-3.3.6/bin/mapred job  -kill job_1792049490106_0002
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-10-15 13:03:36,692 Stage-1 map = 0%,  reduce = 0%
2026-10-15 13:03:43,069 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 3.67 sec
2026-10-15 13:03:48,296 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 6.35 sec
MapReduce Total cumulative CPU time: 6 seconds 350 msec
Ended Job = job_1792049490106_0002
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 6.35 sec   HDFS Read: 834095 HDFS Write: 302 SUCCESS
Total MapReduce CPU Time Spent: 6 seconds 350 msec
OK
Song of Songs: The Journey of a Consecrated Virgin      2020    9.7
Beyond the Smoke        2022    9.7
America's Core  2008    9.6
Kot ptic        2006    9.6
Man's Best Friend       2012    9.6
Time taken: 27.777 seconds, Fetched: 5 row(s)
```

*Figure 4.8: Successful Hive query execution displaying the five highest-rated movies.*

## Movie Recommendation Output

The recommendation interface allows a user to choose a title and request similar movies. The captured example selects “A Man and a City” and returns five titles sharing at least one genre, with rating and vote count shown alongside release year and genre. The output includes Time of Violence (Drama, rating 9.0) and other Drama-containing titles. Since matching uses any shared genre, recommendations can share only part of the selected movie’s genre set.

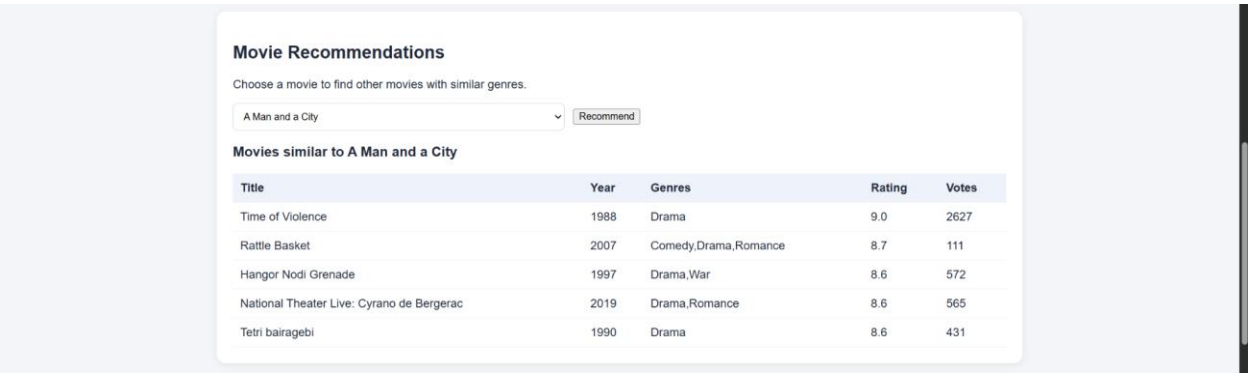



*Figure 4.5: Genre-based movie recommendation results.*

## Key Findings

- The dataset contains 10,000 movie records, supporting a compact but meaningful catalog-analysis demonstration.
- The average rating shown by the dashboard is 6.12; the rating histogram provides a visual view of the rating distribution.
- Release-year counts are uneven and rise substantially in recent years in the displayed chart. The latest years may be partial and should be interpreted cautiously.
- The language query output is dominated by `en` and `Unknown`, so language metadata quality and completeness are relevant considerations.
- The top-rated list combines rating with a minimum vote count, while the recommendation feature uses genre overlap plus a 100-vote threshold.
- The Hive selector makes stored query outputs inspectable through the dashboard, though it does not execute live Hive queries from the browser.

# CHAPTER 5: CONCLUSION AND FUTURE SCOPE

## Conclusion

The project demonstrates an end-to-end movie analytics prototype that connects a structured MySQL dataset, a Flask-based dashboard, statistical visualizations, stored Apache Hive query outputs, and a simple genre-based recommendation feature. The dashboard summarizes the catalog through total movie count, average rating, language diversity, and release-year coverage. The charts and tables provide complementary views of release trends, genre combinations, rating distribution, and highly rated movies.

The recommendation component turns the catalog into a basic discovery tool by finding movies with overlapping genres and ranking them by their existing ratings and vote counts. It is intentionally simple and explainable, making it suitable for demonstrating how SQL-backed data can support a user-facing feature. The project also illustrates how results from separate analytical tools can be brought into one interface. The current Hive section uses saved query results, so a live Hive integration is a possible extension rather than a capability claimed by the present dashboard.

## Future Scope

- Connect the dashboard to Hive through a controlled service so selected queries can run live rather than reading saved JSON results.
- Document the dataset source, schema, provenance, licensing, and refresh procedure.
- Improve recommendations with genre-weighted similarity, a rating-confidence score, or collaborative filtering if user-rating data becomes available.
- Add search, pagination, filters for release year/language, and more responsive chart layouts.
- Normalize the genre field so analysis can count individual genres as well as genre combinations.
- Add data-quality checks for missing years, language codes, duplicate titles, and outlier values.
- Automate the R visualization generation and ensure the dashboard clearly indicates when chart images were last refreshed.
- Deploy the application with environment-based database credentials, validation, error handling, and access controls.

# APPENDIX: DEMONSTRATION CHECKLIST

- Open the Flask dashboard and explain the four summary cards.
- Explain the release-year, genre-combination, and rating-distribution visualizations.
- Show the top-rated movies table and explain the 1,000-vote threshold.
- Select each saved Hive query result and point out the displayed SQL and result rows.
- Select a movie in Movie Recommendations and explain genre overlap, the 100-vote threshold, and rating-based ordering.
- Clarify that the Hive results are stored outputs and that recommendations are genre-based, not personalized machine learning.